

Checklist de Requisitos — Tech Challenge Fase 4

Monitoramento Multimodal de Pacientes Hospitalares (áudio · vídeo · texto)

Repositório Git

(inserir URL do GitHub)

Vídeo demonstrativo

(inserir URL YouTube/Vimeo)

Integrantes

Este documento mapeia **cada requisito do enunciado** para a implementação correspondente, com instruções de verificação para a banca. Todos os comandos assumem Python 3.10+ executado na raiz do projeto, após `pip install -r requirements.txt`. Legenda: ✓ implementado e verificado · ☐ pendente.

1 Análise de Vídeo (RF01)

✓	REQUISITO DO ENUNCIADO	IMPLEMENTAÇÃO	COMO VERIFICAR
✓	Processar vídeos clínicos (fisioterapia / cirurgias gravadas)	<code>video_analysis/video_pipeline.py</code> — <code>VideoAnalysisPipeline.process(video)</code> decodifica qualquer formato suportado pelo OpenCV	<code>VideoAnalysisPipeline().process('data/samples/video_teste.mp4')</code>
✓	Detectar movimentos / eventos fora do padrão esperado	Índice de movimento por quadro + detecção de imobilidade prolongada e picos abruptos (<code>anomaly_detection/movement.py</code>), reutilizado pelo pipeline de vídeo	Achados retornados no <code>ModalityResult</code> da chamada acima
✓*	Análise postural — enunciado sugere “modelos como OpenPose”	MediaPipe Pose (33 landmarks corporais) em <code>video_analysis/pose_analyzer.py</code> ; extrai <code>movement_index</code> e <code>trunk_angle</code> por quadro, com regra de desvio postural $\geq 35^\circ$	Vídeo com paciente na horizontal gera “Desvio postural: inclinação de tronco de $\sim 90^\circ$ ”
✓	YOLOv8 para detecção de objetos e áreas críticas	<code>video_analysis/object_detector.py</code> via <code>ultralytics</code> (YOLOv8n), com amostragem configurável de quadros	<code>ObjectDetector().detect_video(video, sample_rate=30)</code>
✓	Gerar relatórios automáticos indicando desvios ou falhas	Cada análise produz <code>ModalityResult</code> com sumário e achados tipados (<code>Finding</code> : severidade, descrição, score, timestamp, metadados), serializados em JSON pela API	<code>POST /intake</code> com upload de vídeo → campo <code>modalities.video</code>

* **Substituição justificada** (detalhada no relatório técnico, §3.1): o OpenPose exige compilação C++ com CUDA, inviável de reproduzir de forma confiável no ambiente do projeto. O MediaPipe Pose entrega os mesmos landmarks corporais para análise postural com instalação via `pip` e execução em CPU. O contrato interno (`PoseFrame`) é agnóstico ao modelo, permitindo trocar o extrator sem alterar a lógica de detecção.

2 Análise de Áudio (RF02)

✓	REQUISITO DO ENUNCIADO	IMPLEMENTAÇÃO	COMO VERIFICAR
✓	Processar áudios de consultas médicas	<code>audio_analysis/audio_pipeline.py</code> — fluxo áudio → transcrição → insights → achados	Tela <code>/intake</code> : gravar pelo microfone (WAV 16 kHz mono gerado no navegador) ou enviar arquivo

✓	REQUISITO DO ENUNCIADO	IMPLEMENTAÇÃO	COMO VERIFICAR
✓	Detectar alterações indicativas de condições médicas (cansaço, dificuldades respiratórias)	Dicionário clínico pt-BR com severidade por termo em <code>text_analytics.py</code> : “cansaço/fadiga” → MEDIUM; “falta de ar / dificuldade para respirar” → HIGH; “dor no peito” → CRITICAL	<code>TextAnalytics().analyze('estou cansado e com falta de ar')</code>
✓	Azure Speech to Text para transcrever os áudios	<code>audio_analysis/speech_to_text.py</code> — SDK oficial com reconhecimento contínuo em pt-BR; conversão automática de formatos não-WAV via ffmpeg; fallback offline quando sem credenciais	<code>GET /health</code> → <code>capabilities.azure_speech: true</code> ; transcrição real retorna <code>source: "azure"</code>
✓	Identificar termos críticos e sentimentos com Azure Text Analytics	<code>audio_analysis/text_analytics.py</code> — sentimento via Azure Language (<code>analyze_sentiment</code> , pt) e termos críticos por dicionário clínico; fallback léxico offline	<code>GET /health</code> → <code>capabilities.azure_language: true</code> ; painel do dashboard realça os termos por severidade

3 Detecção de Anomalias (RF03)

✓	REQUISITO DO ENUNCIADO	IMPLEMENTAÇÃO	COMO VERIFICAR
✓	Séries temporais de sinais vitais (batimentos, pressão arterial, oxigenação)	<code>anomaly_detection/vitals.py</code> — três técnicas complementares: regras fisiológicas (faixas clínicas de segurança), z-score móvel 3,5σ (desvio da linha de base do próprio paciente) e IsolationForest (padrão multivariado atípico)	<code>pytest tests/test_vitals.py -v</code> · Dashboard → “Executar ciclo”: dessaturação SpO ₂ 96→85%, taquicardia 135 bpm e pico hipertensivo 185 mmHg
✓	Evolução de prescrições (alterações inesperadas no tratamento)	<code>anomaly_detection/prescriptions.py</code> — dose acima da máxima de referência, salto abrupto de dose (> 2×), interações medicamentosas conhecidas e reintrodução de medicamento suspenso	<code>pytest tests/test_prescriptions.py -v</code> · Demo detecta dipirona 1000→3000 mg e warfarina + ibuprofeno
✓	Padrões de movimentação do paciente durante a internação	<code>anomaly_detection/movement.py</code> — imobilidade prolongada (risco de lesão por pressão / TVP) e picos de movimento (queda ou agitação)	<code>pytest tests/test_movement.py -v</code>
✓	Gerar alertas automáticos para a equipe médica	<code>integration/fusion.py</code> (fusão multimodal ponderada + bônus de corroboração → score de risco → <code>Alert</code>) e <code>integration/alerts.py</code> (três canais: console, arquivo JSONL e webhook Teams/Slack), com campo <code>hypotheses</code> de apoio à decisão	<code>python scripts/run_demo.py</code> → alerta CRÍTICO no console; com <code>ALERT_CHANNEL=webhook</code> o alerta chega ao Teams/Slack

4 Requisitos transversais

✓	REQUISITO	IMPLEMENTAÇÃO
✓	Fusão multimodal (texto + áudio + vídeo)	<i>Late fusion</i> com pesos clínicos por modalidade (vitalis 1,0 · prescrições 0,9 · vídeo 0,7 · áudio 0,6) e bônus de corroboração — múltiplas modalidades concordando elevam o score de risco (<code>integration/fusion.py</code>)
✓	Integração com serviços em nuvem (Azure Cognitive Services)	Azure Speech to Text e Azure Language (Text Analytics), com credenciais lidas de <code>.env</code> — nunca versionadas; <code>.env.example</code> documenta cada variável
✓	Alertas em tempo real (ou <i>near real-time</i>)	API REST FastAPI (<code>/monitor</code> , <code>/monitor/vitals</code> , <code>/monitor/prescriptions</code> , <code>/intake</code>) com despacho imediato por webhook

✓ REQUISITO	IMPLEMENTAÇÃO
✓ Datasets (PhysioNet / Google AudioSet sugeridos)	Dados sintéticos reprodutíveis com seed fixa e anomalias plantadas em posições conhecidas (<code>synthetic.py</code>), garantindo verificação determinística de cada detector. Os detectores aceitam qualquer <code>DataFrame</code> , WAV ou MP4 no formato documentado em <code>data/README.md</code> , permitindo alimentar dados do PhysioNet/AudioSet sem alteração de código
✓ Código organizado em Git com histórico de commits coerente	~20 commits seguindo <i>Conventional Commits</i> , cobrindo scaffolding, cada requisito funcional, testes, documentação e correções

5 Entregáveis da Fase 4

✓ ENTREGÁVEL	LOCALIZAÇÃO
✓ Código-fonte completo em repositório Git	Este repositório, com histórico de commits coerente e CI configurado
✓ Relatório técnico — fluxo multimodal, modelos aplicados a cada tipo de dado, resultados e exemplos de anomalias detectadas	<code>docs/relatorio_tecnico.md</code> — arquitetura (§2), modelos por modalidade (§3), fusão (§4), resultados com tabela de anomalias (§6), limitações (§8)
☐ Vídeo demonstrativo (até 15 min, YouTube/Vimeo público ou não listado)	<i>Inserir URL após gravação — roteiro sugerido no relatório técnico, §9</i>

Cobertura exigida no vídeo demonstrativo

✓ ITEM EXIGIDO PELO ENUNCIADO	COMO DEMONSTRAR
☐ Exemplo prático da análise de áudio e vídeo	Tela <code>/intake</code> : gravar a voz no microfone (transcrição Azure com termos críticos realçados) e enviar um vídeo real (pose + YOLOv8)
☐ Detecção e resposta a anomalias	Dashboard: alternar entre os cenários “estável” e “crítico”, mostrando o score de risco e os achados mudando
☐ Integração dos serviços Azure	<code>GET /health</code> exibindo <code>azure_speech: true</code> e <code>azure_language: true</code> ; transcrição retornando <code>source: "azure"</code>
☐ Fluxo final do alerta à equipe médica	Alerta CRÍTICO no banner do dashboard, com hipóteses de apoio à decisão e (opcional) chegada no webhook do Teams/Slack

6 Extras implementados (além do exigido)

- **Dashboard clínico web interativo** (`/`) — medidor de risco multimodal, gráficos de sinais vitais, feed de achados filtrável por severidade e painel de transcrição com realce de termos críticos.
- **Coorte de 20 pacientes** (`/patients`) — seleção com busca/filtro e análise automática “amarrada” ao ID (vitalis + áudio + vídeo + prescrições), com **player de vídeo real** e **áudio TTS em pt-BR** por paciente.
- **Tela de captura clínica** (`/intake`) — formulário multimodal com gravação ao vivo de microfone e webcam, upload de arquivos e resultado renderizado imediatamente.
- **Hipóteses interpretativas no alerta** — camada de apoio à decisão por regras (formato “compatível com: ...”), com aviso explícito de que não substituem avaliação médica; combina achados de múltiplas modalidades (ex.: dispneia relatada + dessaturação objetiva).
- **Modelagem clínica calibrada** — significância estatística e clínica (z-score com variação absoluta mínima; IsolationForest restrito a sinais fora da faixa normal): estável → 0,00, crítico → 1,00.
- **Degradação graciosa** — sem GPU ou credenciais Azure, todo o pipeline executa em modo offline, garantindo que a correção seja reprodutível em qualquer máquina.
- **Deploy em nuvem** — `Dockerfile` e guia para Azure Container Apps (`docs/deploy_azure.md`).

- **Qualidade de engenharia** — 41 testes automatizados (pytest), lint com ruff, verificação de tipos com mypy e CI no GitHub Actions para Python 3.10, 3.11 e 3.12.

7 Guia rápido de correção (5 minutos)

```
# 1. Setup do ambiente
python -m venv .venv && .venv\Scripts\activate      # Windows
pip install -r requirements.txt

# 2. Suíte de testes automatizados (41 casos)
pytest

# 3. Demonstração ponta-a-ponta no terminal
python scripts/run_demo.py
# -> alerta CRÍTICO com 4 modalidades corroborando, score 1.00

# 4. Dashboard, captura clínica e documentação da API
uvicorn multimodal_monitor.api.main:app --app-dir src
# -> http://127.0.0.1:8000/          dashboard de monitoramento
# -> http://127.0.0.1:8000/patients coorte: escolher paciente e analisar
# -> http://127.0.0.1:8000/intake   captura clínica multimodal
# -> http://127.0.0.1:8000/docs     Swagger UI
```

Sobre a integração Azure: para exercitar os serviços reais, copiar `.env.example` para `.env` e preencher as chaves do Azure Speech e do Azure Language (instruções detalhadas no README). Sem as credenciais, o sistema opera em modo offline *sem perda de funcionalidade demonstrável* — todos os requisitos permanecem verificáveis.